CLOUD FUNCTION SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

© Shubham Wa

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

1. A file lands in a GCS bucket and must automatically trigger downstream ETL. How would you design this with Cloud Functions?

I'd use a Cloud Function as the simple, fast trigger not the heavy worker. The function subscribes to the GCS OBJECT_FINALIZE event, does quick validation (filename pattern, size, basic header check) and extracts metadata like generationId and producer checksum. If the file clearly fails these quick checks, move it to a quarantine prefix and emit a metric/alert so ops can investigate; don't try to salvage big failures inside the function.

For real ETL work, hand the job off. I'd publish a message to Pub/Sub containing the object path, generationId, checksum and any parsed metadata, or directly kick off a Dataflow job for very large files. That decouples ingestion from processing, allows the heavy transform to scale independently, and prevents the function from running long or hitting memory/time limits. Use a manifest (BigQuery or Firestore) or the object generationId to ensure idempotency downstream workers should check the manifest before processing.

Keep the function minimal and safe-to-retry: use a least-privilege service account, set conservative timeouts and max-instances, and write structured logs with a correlation id so you can trace the file end-to-end. Emit metrics like file-count, bytes, and validation failures so SREs can create alerts and dashboards. Also verify producer CRC/MD5 if the producer provides it in metadata to catch upload corruption early.

Plan DLQ and quarantine flows: if publishing to Pub/Sub fails repeatedly, push the event to a dead-letter topic with error context; for file-level issues, copy to a quarantine bucket with error metadata. Automate a small replay capability so fixed files can be reprocessed without manual copying. Have an automated end-to-end test that uploads a known file and verifies the manifest and downstream result.

Finally, choose the downstream engine per workload: Dataflow (Beam) for large, complex transforms and joins, Cloud Run for custom libraries or moderate transforms, and BigQuery load jobs for batch loads. The Cloud Function's job is to validate and hand the baton to the correct worker so the pipeline remains simple, observable, and resilient.

2. Your team needs to run lightweight transformations whenever a Pub/Sub event arrives. How would you set it up?

I'd use a Pub/Sub-triggered Cloud Function as the primary consumer because it's serverless, autoscaling, and ideal for small, fast transforms. The function should do minimal work: validate and normalize the JSON, enrich with ingest metadata if needed, and write the result to the next sink BigQuery, another Pub/Sub topic, or GCS. Keep it stateless so it can scale horizontally without coordination.

Handle acking and retries carefully: don't ack until the transform and downstream write succeed. Configure the subscription with a sensible retry policy and a Dead Letter Topic (DLQ) so poison messages or schema mismatches don't spin forever. When a message goes to DLQ, include an enriched envelope (original payload, error reason, function version) to make troubleshooting and replay easier.

For throughput bursts or cost efficiency, batch writes: either buffer a small batch in-memory (flush on N rows or after T ms) or push transformed records to a staging topic/GCS and have a batch writer (Cloud Run or Dataflow) do bulk writes to BigQuery. If sustained high throughput is expected, prefer Dataflow or Cloud Run with explicit batching and backpressure control Cloud Functions are fine up to a point but offer less low-level control.

Make everything idempotent: attach message_id or an ingest_id and dedupe on the BigQuery side (MERGE from staging or use dedupe queries). Add tracing/correlation ids so a single

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

message can be followed across services, and monitor processing latency, DLQ volume, and Pub/Sub backlog to detect issues early.

Secure the pipeline with least-privilege service accounts for Pub/Sub/BigQuery/GCS, and keep a runbook for DLQ triage and replays. Finally, document schema contracts with producers to minimize schema drift and regularly surface DLQ samples to the producers for quicker fixes.

3. Business requires a BigQuery summary table to be updated when new data arrives. How would you orchestrate this with Cloud Functions?

I'd keep the Cloud Function as the orchestrator/lightweight trigger and push heavy aggregation to BigQuery or a batch job. When new data arrives, the function validates the event and either performs a small, cheap update (for trivial increments) or writes normalized rows to a staging table/GCS and triggers a BigQuery load or a Dataflow job for heavier aggregation. That keeps the function responsive and avoids long-running work inside serverless.

For near-real-time small updates, streaming inserts into a staging table followed by an idempotent MERGE into the summary table works well MERGE keyed by natural keys plus ingest_time prevents duplicates. For larger or windowed aggregations, write transformed files to GCS and start a GCS→BigQuery load job or kick off Dataflow, then MERGE results into the summary table. Prefer batch load jobs for large volume (cheaper and transactional) and streaming for low-latency low-volume needs.

Enforce idempotency by tracking the source object generationId or message_id in an ingest manifest table; use MERGE statements so replays don't create duplicates. Use partitioned and clustered tables for the summary table to keep query cost low and performance high, and include audit columns (ingest_time, source_id, process_id) to enable tracing and reprocessing for specific ingests. Label BigQuery jobs for cost tracking and include job labels in logs.

If sub-minute freshness isn't required, I'd recommend a simpler approach: Cloud Function publishes an aggregation request to a topic and a scheduled runner (Cloud Scheduler + Cloud Run or a BigQuery scheduled query) computes the summary every few minutes. That reduces operational complexity and gives predictable cost and SLA. For strict real-time needs accept the added complexity but ensure monitoring for latency, failures, and backlogs.

Secure and operationalize: give the orchestrator least-privilege, set BigQuery job quotas and retry/backoff patterns, and maintain a clear replay procedure (re-run MERGE for a given ingest_id). In an interview I'd close by weighing trade-offs real-time complexity vs operational simplicity and recommend batching when business tolerance allows.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

4. A CSV file lands in a raw bucket and needs to be validated, transformed, and loaded to BigQuery. How would you implement this?

I'd keep the ingestion pipeline simple and reliable: a Cloud Function triggered on GCS OBJECT_FINALIZE does quick, deterministic validation first check filename pattern, size, encoding (UTF-8), and validate headers against an expected schema by parsing only the first few rows. If those checks fail, move the file to a quarantine prefix and emit a metric and alert so the data team can inspect; don't try heavy fixes in the function.

Once the file passes quick checks, the function should decide the execution path based on file size and transformation complexity. For small, trivial transforms (column renames, type coercion, small lookups) it's fine to transform inside the function and write a Parquet/ndjson file to a staging bucket. For medium-to-large transforms joins, windowing, or anything needing scale publish a message to Pub/Sub and let Dataflow or a Cloud Run batch job do the heavy lifting. This keeps the function fast and the system resilient.

For loading into BigQuery prefer batch load jobs from GCS for larger files because they are cheaper and atomic. The transform job should write to a staging table or staging files in GCS, then trigger a GCS→BigQuery load job. For near-real-time low-volume use-cases, consider buffered streaming inserts in batches and then a periodic MERGE into the target table to dedupe. Always use partitioned/clustered target tables to keep costs down.

Make everything idempotent: record object name + generationId + ingest_time in a manifest table before processing, and check that manifest at start so retries don't double-process. Use MERGE statements into the final table keyed by natural keys and ingest_id to handle duplicates safely. Add audit columns (source_file, ingest_time, process_id) so you can trace back and reprocess specific ingests if needed.

Operational pieces: include structured logs and correlation IDs, emit metrics (validation failures, processing latency, bytes processed), and set up alerts for DLQ or high failure rate. Maintain a small replay mechanism to re-enqueue quarantined/fixed files and a runbook for schema evolution e.g., accept new nullable columns by default and require producer change requests for breaking changes.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

5. Some files uploaded to GCS are corrupt. How would you detect and handle them with Cloud Functions?

Detect early with cheap checks in the trigger function: compare producer checksums if available (crc32c/md5), verify object size against expected limits, confirm UTF-8 encoding, validate headers and attempt to parse a small sample of rows. If the parser fails or types are clearly wrong in the sample, mark the file as corrupt failing fast prevents wasted downstream compute.

When corruption is detected, isolate the file automatically: copy it to a quarantine bucket/prefix with metadata that records the error reason, original generationId, and detection time. Also write a record to a processing manifest or error table with a sample of the rejected rows and the parser error to help debugging. Publish an alert (Slack/email) with a link to the quarantined file so the data producer or on-call can investigate.

Don't immediately give up on transient issues implement a short automated retry policy for uploads (for example, one retry after a configurable delay) because network glitches can produce temporary corruption. If the file fails again after retries, keep it quarantined and escalate. Keep the retry count low to avoid clogging pipelines with perpetually bad files.

If you can salvage partial data, consider row-level rejection: write good rows to a "partial-good" staging file or table and write rejected rows with error reasons to a rejects table or file. This lets business teams recover usable data quickly while still surfacing corrupt records for root-cause analysis. Otherwise, prefer full quarantine to avoid silent bad data in production.

Operationalize by collecting metrics on corrupt files (rate, top producers, common error types), a replay capability for fixed/quarantined files, and a runbook for triage. Where possible, enforce producer-side checksums or signing so corruption is detectable at ingest time and producers can be held to a contract for reliability.

6. You need to move processed files from the raw zone to an archive bucket. How would you design the workflow?

Move files only after a deterministic success marker either a BigQuery job completion, Dataflow job success, or an explicit processed flag written to a manifest table. Don't rely on file age alone. The worker that completes processing should emit the "processed" event (manifest update or Pub/Sub message) that triggers the archival step.

For the actual move perform copy-verify-delete: have a Cloud Function or Cloud Run job copy the object to the archive bucket (preserving metadata and ACLs), verify the copy by comparing crc32c or size, and only then delete the source. That pattern avoids accidental data loss from partial or failed copies and is safe to retry. Log checksum and job_id in an audit table for traceability.

Make the move idempotent and manifest-driven: consult the manifest table before moving (check generationId) and skip files already archived. If you prefer batch operations, use Cloud Scheduler to run a periodic job that scans the manifest for processed-but-not-archived files and moves them in chunks batching helps with throughput and retry handling.

Apply lifecycle and cost policies on the archive bucket: use Coldline/Archive storage classes and configure lifecycle rules for further retention/expiration. Also tag archived objects with ingestion metadata so downstream teams can find them easily. Keep strict IAM so only the mover service account has delete rights on the raw bucket.

Finally, instrument everything count moved files, copy failures, and backlog size; alert on copy failures or growing lists of processed-but-not-archived files. Keep a manual re-run tool or small admin UI for one-off repairs and document the replay steps in the runbook so on-call can handle edge cases without manual GCS fiddling.

© Shubham Wadekar

7. Your Pub/Sub topic receives thousands of JSON messages per second. How would you use Cloud Functions to process and write them to BigQuery?

I'd treat Cloud Functions as the lightweight, stateless transformer and put durable buffering/aggregation where throughput matters. The function should do quick validation and normalization (parse JSON, coerce types, add ingest metadata and correlation id) and avoid per-message BigQuery calls. Instead it should batch rows either buffer small batches in-memory (flush on N rows or T ms) or write transformed NDJSON/Avro files to a staging GCS location and then trigger a load job. Batching reduces API calls, cost, and contention against BigQuery.

For very high sustained throughput I'd prefer using the BigQuery Storage Write API or a batching worker (Cloud Run/Dataflow) rather than raw streaming inserts from many function instances. One good pattern is: Cloud Function validates and publishes to a high-throughput intermediate topic or staging bucket, and a dedicated batch writer (Cloud Run/Beam) reads batches and writes to BigQuery efficiently. This gives you control over concurrency, backpressure, and retries.

Make everything idempotent: attach message_id or a generated ingest_id and include that in the row; use a staging table + dedupe MERGE into the target table or write with the Storage Write API with dedupe keys. Also tune function concurrency (max-instances) to protect BigQuery quotas and avoid sudden spikes. If you must stream directly from functions, do batched streaming inserts (e.g., 500–5000 rows per request) rather than one-row inserts.

Operationally, monitor Pub/Sub backlog, function error rate, BigQuery insert errors and quota usage. Configure appropriate retry/backoff for transient errors and use DLQs for poison messages. Add tracing so a single message's lifecycle (Pub/Sub → function → BigQuery) is easy to follow and set alerts for rising DLQ rates or growing Pub/Sub backlog.

Secure the pipeline: give the function and writers least-privilege service accounts for Pub/Sub/GCS/BigQuery, label jobs for cost tracking, and keep a replay mechanism so messages in staging/DLQ can be reprocessed after fixes.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

8. Some messages are failing processing due to schema mismatch. How would you handle retries or DLQs?

First, classify failures as transient (downstream unavailability) or permanent (schema mismatch or poison data). For transient errors rely on Pub/Sub's retry redelivery with exponential backoff; for permanent schema mismatch avoid infinite retries and route the message to a Dead Letter Topic (DLQ) after a small retry count so the pipeline isn't clogged by poison messages. Configure the subscription with a DLQ and a sensible maxDeliveryAttempts.

Enrich DLQ messages with context so they're actionable: include the original payload, error type, function version, schema expected, and a small sample of the parsed fields or stack trace. Store DLQ samples in a BigQuery "rejects" table or a GCS folder and surface them on a dashboard for the data producers and on-call. That makes root-cause analysis and producer fixes fast.

Don't treat all schema mismatches the same implement tolerant parsing where safe (coerce strings to numbers with null-on-fail, accept extra nullable fields) so non-breaking changes don't cause rejections. For true breaking changes, alert the producer and block only the affected messages in the DLQ until a contract change or a transformation mapping is applied. Provide a simple replay tool that can reprocess fixed DLQ messages after remediation.

Optionally implement partial-accept: write the valid fields to the good sink and rejected fields to a rejects table with reasons, so downstream consumers can still get usable data while problematic fields are investigated. But use this sparingly partial acceptance complicates downstream logic and SLAs.

Operational items: track DLQ rate and top error reasons, keep a runbook for DLQ triage and replay, and version your schema contracts so producers and consumers can coordinate changes without breaking production.

9. One Pub/Sub event must trigger multiple downstream actions (BigQuery load, GCS write). How would you design this fan-out pattern?

Prefer native Pub/Sub fan-out: create multiple subscriptions on the same topic and have each subscription trigger its own single-responsibility consumer (one Cloud Function writes to BigQuery, another writes to GCS, etc.). This keeps each consumer independent, scales per consumer, and isolates failures so one downstream action doesn't block others. Correlate work using a shared correlation id in the original message so logs and traces can be stitched together.

If you need transformation or routing control before fan-out, use one lightweight orchestrator function that subscribes to the source topic and republishes tailored messages to dedicated downstream topics (e.g., topic-bigquery, topic-gcs). That gives you the ability to enrich or split the payload and to add per-target routing logic, at the cost of one extra hop. Keep the orchestrator minimal to avoid a single point of failure.

Make each downstream consumer idempotent and give each subscription its own DLQ so failures are contained and replayable independently. Use per-consumer monitoring (success rate, latency, DLQ) and an end-to-end dashboard that shows the status of all consumer paths for a given correlation id. This makes troubleshooting much faster.

For strict atomicity (rare and complex) implement a durable staging pattern: orchestrator writes intent and payload to a manifest/staging area (GCS/BigQuery) and downstream tasks read from that manifest and report completion; use compensating actions on failure. Only choose this if the business demands all-or-nothing semantics otherwise the independent-subscriber model is simpler and more robust.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, document the message contract and correlation id rules for producers and consumers, secure each subscription with least-privilege service accounts, and apply resource quotas to prevent noisy neighbors from starving other consumers.

10. You need to run a parameterized BigQuery query daily when a new file lands in GCS. How would you design this pipeline?

Start by treating the file arrival as the signal that creates or updates the set of parameter values you need to run daily. Have a Cloud Function subscribe to the GCS OBJECT_FINALIZE event; the function does quick validation (filename pattern, generationId) and extracts the parameters (file path, date, vendor id, whatever) and writes a single row into a small "scheduled_jobs" table in BigQuery (or a lightweight Firestore collection) that represents "run this query daily for these params." This keeps the arrival handler tiny and durable.

Use Cloud Scheduler to run a daily driver that triggers a Cloud Function (or Cloud Run). The driver reads the scheduled_jobs table and for each entry invokes a parameterized BigQuery job (using the BigQuery API) with the stored parameters. That separation makes scheduling predictable and avoids dynamically creating many scheduler entries everything is driven from a single daily cron that iterates the config rows. If you need per-file daily schedules starting at file arrival time, add a start_date column and let the scheduler only pick rows whose window is active.

When running the parameterized query, prefer submitting an asynchronous BigQuery job (jobs.insert) with query parameters and job labels for tracing and cost allocation. Make those queries idempotent: write results to a partitioned staging table or use MERGE into the final summary table keyed by ingest_id or file generationId so re-runs are safe. Also tag each job with a correlation id so logs and monitoring can connect the daily run back to the originating file.

For durability and replayability, have the Cloud Function that wrote the scheduled row also publish a message to a "one-time-run" Pub/Sub topic if you need an immediate run in addition to the daily schedule. Expose a small admin endpoint (Cloud Run) or use a replay script that allows operators to re-run queries for a given file/param if something fails. Implement retry/backoff for transient BigQuery errors and alert on repeated failures.

Finally, secure and monitor: give the scheduler-runner a least-privilege service account that can read the jobs table and start BigQuery jobs, collect metrics (jobs started, failures, duration), and add alerting on failed job counts or long-running queries. This approach separates concerns (ingest, schedule, execution), is simple to reason about, and is easy to operate and debug in interviews.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. A streaming insert from Cloud Functions to BigQuery is failing due to invalid rows. How would you fix it?

First, capture and examine the row-level errors returned by the BigQuery streaming API those messages tell you whether the problem is type coercion, missing required fields, or unknown fields. Don't ignore insert errors; implement code in the function that inspects BigQuery's response, logs the failing rows with the error reasons, and routes them to a rejects store (GCS/BigQuery rejects table or DLQ) for later triage and replay. That immediate visibility is critical.

Next, add defensive schema validation before attempting the stream insert. Build a small validation layer in the Cloud Function: check required fields, coerce/canonicalize types (string → timestamp, numeric parsing), trim/normalize strings, and apply sensible defaults for optional fields. If a row still fails validation, don't send it to BigQuery push it to a "bad-rows" DLQ with the validation error and the original payload so producers or engineers can inspect. This prevents poison rows from repeatedly failing the stream.

If the workload tolerates it, use the BigQuery streaming insert options or load-job alternatives: for loads from GCS you can set skipInvalidRows or ignoreUnknownValues on load jobs, or use the Storage Write API which gives better throughput and explicit dedupe keys. For high-volume or complex schemas prefer writing validated batches to GCS and kicking off a GCS→BigQuery load job loads are more tolerant/configurable and cheaper per GB than per-row streaming.

Make your insertion idempotent by including an insert_id or ingest_id with each row (or persist message_id). On the BigQuery side use staging + MERGE or dedupe queries to avoid duplicates on retries. Also add monitoring: track rejected-row counts, top error reasons, and latency. Configure alerting for spikes in reject rate so schema drift from producers is caught early.

Finally, create a simple replay process that reads the rejects store, applies fixes or transformations, and re-submits rows once the schema is corrected and document the runbook so on-call can triage quickly. These steps fix immediate failures, reduce future incidents, and make the pipeline resilient to producer-side issues.

12. After inserting data, a summary aggregation query must run. How would you orchestrate both steps with Cloud Functions?

Treat the insertion and aggregation as two discrete, observable steps and avoid doing heavy aggregation inside the trigger function. The Cloud Function that consumes the event should validate and write raw/normalized rows to a staging table (or to GCS), and then emit a lightweight completion event either update a manifest row in BigQuery or publish a message to a dedicated Pub/Sub topic like ingest-complete. That keeps the ingest path fast and reliable.

Have a separate consumer (Cloud Function, Cloud Run, or a scheduled job) subscribe to ingest-complete. That consumer runs the aggregation logic: it either submits an asynchronous BigQuery query/job that reads the staging data and writes to the summary table (MERGE into partitioned table), or it invokes a scheduled BigQuery job if aggregation can be batched. Running the aggregation as a separate job decouples failures and makes retries simpler you can retry the aggregation without re-ingesting raw rows.

Use BigQuery's asynchronous jobs so the aggregator can submit the job and either poll for completion with exponential backoff or leverage job-completion notifications (BigQuery can publish job status to Pub/Sub via job notifications) so you don't block serverless compute while waiting. Ensure the aggregation is idempotent by keying by ingest_id or generationId and using MERGE semantics so re-running the aggregation won't double-count.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Implement end-to-end tracing and monitoring: correlation ids across ingest and aggregation, metrics for insert latency and aggregation duration, and alerts for failed aggregation jobs. Keep an audit table that records ingest_id → insert_status → aggregation_job_id → aggregation_status so operators can see where anything stalled. Provide a replay path: if aggregation fails, someone can re-publish the ingest-complete message or invoke an admin endpoint to re-run aggregation for specific ingest_ids.

Finally, secure and operationalize it: use least-privilege service accounts, label BigQuery jobs for cost allocation, and keep a runbook for common failures. This two-step, event-driven pattern keeps pipelines reliable, observable, and easy to maintain the kind of answer an interviewer wants to hear.

13. Your Dataflow job must run whenever a new file arrives in GCS. How would you set up a Cloud Function trigger?

I'd use a Cloud Function subscribed to the GCS OBJECT_FINALIZE event on the raw bucket so it fires the moment a file is fully uploaded. Keep the function tiny: validate the filename pattern, check generationId and size limits, and extract the minimal metadata (file path, producer id, checksum) needed to start processing. Don't do heavy transforms in the function its role is the trigger and quick gatekeeper.

Once validated, the function should hand off to a durable runner rather than run Dataflow inline. The usual pattern is to publish a message to a dedicated Pub/Sub topic (or call the Dataflow REST API to launch a template) containing the file path and any metadata. Publishing to Pub/Sub decouples failures and gives you retry/delivery semantics; launching a Dataflow template directly is fine when you want immediate start and you handle idempotency carefully.

I prefer using Dataflow templates (Flex Templates for custom images) so the function only needs to kick a template run with runtime parameters (stagingLocation, inputFile, outputTable, jobName). Templates keep the function simple and avoid packaging Beam dependencies into the function. Make sure the function's service account has dataflow.jobs.create and write access to the staging bucket. Also set region and worker options in the template invocation so jobs run in the right place.

Add defensive production controls: check a processing manifest (BigQuery/Firestore) first so you don't launch duplicate jobs for the same object generationId, and publish a small "job-request" record to the manifest before launching. Emit structured logs and a correlation id so you can trace the GCS event → function → Dataflow job end-to-end.

Finally, instrument and test it: verify producer checksums if available, add alerts on stuck or repeatedly failed triggers, and include a simple replay mechanism (re-publish to the Pub/Sub topic or call an admin endpoint) so operators can re-run jobs for specific files without manual GCS copying.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. A Dataflow job requires dynamic parameters (file path, date). How would you pass them from Cloud Functions?

Pass parameters as runtime arguments to the Dataflow template or as configuration in a Pub/Sub message that the Dataflow job reads. The straightforward approach is to invoke a Flex Template run and include a parameters map for example {"inputFile":"gs://bucket/path","processingDate":"2025-09-12","outputTable":"proj:ds.tbl"} so the template receives them as --inputFile, --processingDate, etc. That keeps the contract explicit and avoids ad-hoc config files.

If you prefer a decoupled pattern, the function can publish a Pub/Sub message with the parameters and let a small launcher service (Cloud Run or a Dataflow job that polls) start the job. This is useful when you want centralized retry logic, throttling, or to aggregate parameters before launching. Either way include a correlation id, object generationId and a manifest entry so the run is traceable and idempotent.

Make sure the parameters are validated and sanitized in the Function before sending verify date formats, check file existence (optional quick head check), and limit parameter sizes. Store the parameters (and requested run state) in a manifest table (BigQuery/Firestore) before launching the job so you can prevent duplicates and support replays. Also set meaningful job labels (e.g., source_file, ingest_id) when starting the job so you can filter logs and billing by run.

If secrets or credentials are needed by the Dataflow job, avoid embedding them in the parameters instead give the Dataflow worker service account appropriate access (or use Secret Manager references) so the template can read secrets at runtime. Validate IAM: the Cloud Function needs permission to launch Dataflow jobs and to write the manifest; Dataflow needs access to staging, temp, and output resources.

Finally, test parameter edge-cases and implement a small admin UI or script to re-run jobs with modified parameters for troubleshooting. This makes the pipeline operable and reduces on-call friction when a parameter typo needs a quick replay.

15. Your Dataflow job fails occasionally. How would you design Cloud Functions to handle retries or notifications?

Design the trigger and orchestration so Cloud Functions don't blindly re-launch jobs; instead make them manifest-driven and stateful. When the function launches a Dataflow job, write a manifest record with ingest_id, job_id, status=PENDING and increment attempts. Dataflow job state changes (success/failure) should be captured via Dataflow job notifications (Dataflow can publish job state changes to Pub/Sub) or by polling the Dataflow API from a small watcher service. That way retries are decisioned by state, not blind re-trigger.

On failure, use a controlled retry policy: if the job failed due to transient errors (quota, worker preemptions, temporary upstream issues) and attempts < max_attempts, automatically relaunch with exponential backoff and increased worker resources if applicable. Record each attempt in the manifest and attach failure reason. For persistent or business-data errors (bad input, schema mismatch), send the job to a quarantine workflow and notify the data owner instead of retrying. Don't let retries loop forever.

Send actionable notifications: subscribe a Cloud Function (or Cloud Run service) to the Dataflow job-state Pub/Sub topic that inspects job metadata and failure messages and then posts structured alerts to Slack/Email/PagerDuty with the correlation id, file path, error summary, and a link to logs. Include retry instructions and a one-click "re-run" link (Cloud Run admin endpoint) that operators can use if manual intervention is needed. That minimizes context switching for on-call.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Add automated observability: emit metrics (failure rate, retry count, average run time), create alerting policies for repeated failures or long-running retries, and label Dataflow jobs for cost and debugability. Keep idempotency via ingest_id so re-runs don't double-process data. Also implement a DLQ or quarantined-manifest state for items that exceed retry limits so they can be examined separately.

Finally, keep a runbook and a safe replay mechanism. Provide an admin tool that can re-run a job with the same parameters or with adjusted worker settings, and require a human review step if the job failed due to data issues. This combination of manifest-driven launches, job-state-driven retries, clear notifications, and a replay path makes occasional failures manageable and reduces accidental data duplication or resource thrashing.

16. A JSON file arrives in GCS and must be parsed, filtered, and saved as Parquet. How would you do this with Cloud Functions?

Start with the trigger and a cheap safety check: use a Cloud Function on GCS OBJECT_FINALIZE for the bucket so the function runs only after the upload completes. In the function do very small, fast checks filename pattern, size threshold, crc32c if provided, and sample-parsing the first N JSON objects to confirm it's valid JSON. If the file fails these checks move it to a quarantine prefix and emit a metric; don't waste cycles trying to salvage obviously broken files.

Decide where the work should run based on file size and complexity. If the file is small (tens of MB) and the transformation is light (filtering, dropping fields, simple type coercion), the function can parse the file into memory, apply the filter, convert to Parquet with a lightweight library (pyarrow/fastparquet) and write the Parquet file back to a staging GCS path. Keep memory/time budgets in mind and limit input size or stream-process line-delimited JSON to avoid OOM. If the file is larger or processing is heavier, publish a job request to Pub/Sub and hand off to Cloud Run/Dataflow instead.

When writing Parquet, prefer producing columnar formats (Parquet or Avro) to a staging bucket and use a deterministic naming scheme that includes the source object generationId and a correlation id. This makes artifact tracing and replays easy. Also ensure you preserve provenance metadata (source filename, generationId, ingest_time) either in file metadata or as dataset columns. If using pyarrow inside Cloud Functions, package dependencies correctly (use a custom runtime or Cloud Functions 2nd gen with container image) and test end-to-end for memory and latency.

Make the pipeline idempotent: write a manifest entry (BigQuery or Firestore) before doing the heavy work and check it at start so retries don't reprocess the same generationId. After Parquet is written, optionally kick a load job (GCS → BigQuery) or just leave the Parquet in the staging area for downstream consumers. Always verify the written file (size, crc) and update manifest status to SUCCESS, copying the original to archive only after successful verification.

Operationalize with metrics, logs, and alerts: record processing time, bytes processed, error reasons, and number of output files. Add a replay path that re-enqueues the manifest row or republish the Pub/Sub message so operators can re-run a specific object. In short: Cloud Function for small/fast jobs, staged Parquet in GCS, manifest-driven idempotency, and handoff to Cloud Run/Dataflow for anything bigger.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

17. Your transformation logic is too heavy for Cloud Functions. How would you decide when to push processing to Dataflow?

Make the decision on objective, measurable factors: input size, CPU/memory needs, processing time, statefulness, and throughput/latency requirements. If files are consistently large (hundreds of MBs or GBs), transforms require joins/windowing/aggregation, or processing time regularly exceeds function timeouts, that's a clear sign to move to Dataflow. Similarly, if you need exactly-once semantics, event-time windowing, or complex fan-in/fan-out, Dataflow/Beam gives battle-tested semantics you'll want.

Consider operational needs: Cloud Functions are great for simple, stateless, fast tasks and for teams that want low ops. But for heavy transforms you'll need more control over parallelism, autoscaling, worker sizing, checkpointing, and backpressure all areas where Dataflow shines. If you need to tune worker types, use custom containers, or attach GPUs/large-memory machines, serverless functions become awkward and expensive. Dataflow's autoscaling, shuffle, and streaming/batch modes make it the right fit for sustained heavy workloads.

Look at cost and efficiency too. For bulk processing, Dataflow's model (worker VMs doing work in parallel) will often be cheaper and more predictable than many concurrent Cloud Function instances making repeated API calls and hitting egress/IO limits. If you find you are spawning hundreds of function instances to handle a sustained load, that's a cost and quota red flag. Also evaluate failure modes: if partial failures require complex orchestration or checkpoint-based replay, Dataflow's retry semantics and stateful transforms reduce complexity.

Operational signals also matter: if testing shows frequent OOMs, timeouts, or inconsistent performance under load, treat that as a hard trigger to refactor. Start with a pilot: reimplement a representative job in Dataflow (or Cloud Run with batching) and compare run-time, cost, and operational complexity. If Dataflow reduces code complexity and improves reliability, make the migration.

Finally, consider developer velocity and maintainability. If your team needs to reuse Beam pipelines, support windowed analytics, or implement event-time semantics, investing in Dataflow is worthwhile. If the transformation is only moderately larger than function capacity, Cloud Run (containers) can be a middle ground. The rule of thumb: push to Dataflow when scale, state, complexity, or cost pushes you beyond the practical limits of short-lived functions.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

18. You need to enrich records by calling an external API before saving to BigQuery. How would you design this?

Treat external API enrichment as a potentially slow, rate-limited, and unreliable dependency design for batching, caching, and graceful degradation. I'd avoid calling the external API per-record synchronously inside a Cloud Function at high volume. Instead, validate and normalize incoming records quickly, then enrich via a separate enrichment stage that supports batching and retries. For small volumes or low QPS, synchronous calls from Cloud Functions with careful timeouts and rate-limiting can work; for higher volumes use an async pipeline.

A robust pattern is: Cloud Function receives the event, does lightweight validation, then publishes batched enrichment requests to Pub/Sub or writes batches to a staging location in GCS. A worker (Cloud Run or Dataflow) consumes batches, dedups external ids, uses a cache (Redis/MemoryStore or an in-process LRU) to avoid repeated calls, and calls the external API in bulk or in parallel with concurrency limits. This worker applies backoff, circuit-breaker logic, and writes enriched results to a staging table or files for subsequent load into BigQuery. Caching reduces call volume and latency significantly.

Design for partial failure and DLQ: if the external API rejects certain ids or is down, separate permanent failures (bad id) from transient ones (timeout). Route transient failures to retry with exponential backoff; after N attempts move them to a DLQ with the original payload and error reason so you can replay later. For permanent failures, write the row to a "rejected" store with an enrichment error column so you don't silently drop data. Provide a manual replay mechanism for fixed rows.

Security and secrets: store API keys in Secret Manager and mount/access them securely from Cloud Run/Dataflow. Respect rate limits and terms of service of the external API; add metrics for API latency, error rates, and cost. If enrichment includes sensitive PII, ensure encryption in transit and at rest and validate compliance. Also consider TTL-based caching so stale enrichment is refreshed periodically but you don't overwhelm the external service.

Finally, keep the pipeline idempotent and observable: add correlation ids, log enrichment attempts and outcomes, label BigQuery loads for cost tracking, and provide dashboards for enrichment success rate and cache hit ratio. This design isolates enrichment concerns, lets you scale and throttle independently of ingestion, and provides robust handling for the inevitable API failures exactly what an interviewer wants to hear.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

19. You need to validate incoming CSV files against a predefined schema. How would you implement this in Cloud Functions?

Start with a Cloud Function triggered on GCS OBJECT_FINALIZE for the raw CSV bucket so validation runs only after the upload completes. The function should first perform cheap, deterministic checks filename pattern, size limit, and optionally compare producer-provided checksum then open the file as a stream and read only a small sample (first N rows) to sanity-check encoding and headers before full validation. This keeps the function fast-failing on obvious problems.

Keep the schema as a first-class artifact: store the expected schema definition as a JSON/YAML config in a config bucket or a schema registry table (BigQuery / Firestore). The Cloud Function loads that schema (cached in-memory for repeated invocations) and uses it to drive validation rules: required columns, types, nullable flags, allowed enums, max-lengths, and custom regex or date formats. Having the schema externalized makes evolution and testing easy.

Implement validation in two passes where possible. Pass 1 does header and lightweight type checks on the sample to quickly reject malformed files. Pass 2 streams through the file row-by-row (using a robust CSV parser that handles quoted fields and line breaks) and applies per-row validation rules. For performance, process rows in a streaming fashion (no full-file memory load) and emit a small number of example bad rows early to the rejects pipeline so operators get immediate feedback.

Make the function idempotent and manifest-driven: before validating, write a manifest entry (object name + generationId + status=VALIDATING) so retries don't revalidate the same generation. After validation complete, update manifest to VALID or INVALID with counts. Use structured logs and a correlation id so validation runs are traceable end-to-end. If files are very large and validation is expensive, consider handing off the full-file pass to Cloud Run or Dataflow after the cheap sample pass in the function.

Operational considerations: expose a lightweight admin endpoint or CLI to re-run validation for a file, version the schema with timestamps and deprecation notes, and add metrics (validation time, invalid-row-rate, schema-versions-in-use) so SRE and data teams see schema drift early. This approach keeps the Cloud Function simple but reliable and makes schema evolution manageable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

20. If invalid rows are found, you must log them separately in GCS. How would you design the pipeline?

On detecting invalid rows during the streaming validation pass, don't throw them away emit them to a rejects sink immediately. In the Cloud Function stream, collect invalid rows into small in-memory buffers and flush periodically to ndjson or CSV reject files in a dedicated GCS "rejects" prefix (e.g., gs://bucket/rejects/YYYY/MM/DD/<source>-<shard>-<timestamp>.ndjson). Include for each rejected row the original row, the validation error code/reason, and minimal provenance (source file, row_number, generationId, correlation_id) so debugging and replay are easy.

Choose a flush/batching strategy to avoid too many small objects: flush when buffer reaches N rows or after T seconds. Name files deterministically and include a shard id so parallel invocations don't collide. After each flush, write a small manifest entry for the reject file (path, count, earliest_row, last_row) to a rejects index table (BigQuery or Firestore) so UIs and downstream jobs can discover rejects quickly. This gives operators a searchable catalog of rejections.

If you expect high reject volume, write rejects to an intermediate Pub/Sub topic instead and use a dedicated consumer (Cloud Run) to batch-write reject files to GCSthis offloads the main validation function and gives better throughput and backpressure control. Also consider writing a summary row per reject file to BigQuery for fast analytics on error types and producers.

Secure and retention: apply appropriate IAM so only analysts and ops can read rejects, and set lifecycle rules if rejects are only needed short-term. Add metadata tags (schema_version, validation_rule_id) to reject objects for auditability. Provide a simple replay mechanism that reads a rejects file, applies fixes (or manual edits), and re-submits only the corrected rows to the ingest pipeline.

Finally, monitor reject metrics (reject rate per file, top error reasons) and alert on spikes so the producer teams are notified quickly. A small web page or notebook that can fetch example rejects from GCS and show the error reasons will speed triage and is highly appreciated by interviewers and operators alike.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. Business requires a report of validation errors for each batch of files. How would you implement it using Cloud Functions?

Create a small reporting pipeline that aggregates per-file validation results into a batch-level report. Each validation run (the Cloud Function) should write a per-file validation summary to a central table (BigQuery or Firestore) include file name, generationId, schema_version, total_rows, valid_rows, invalid_rows, top_error_types (with counts), rejects_file_paths, start_time, end_time, and correlation_id. These row-level summaries are the atomic inputs for the batch report.

Define what a "batch" means (time-window, folder prefix, or an orchestrator grouping) and make that explicit in the manifest. If files are grouped by day or by job_id, ensure each validation writes the group id into its summary record. Then schedule a lightweight Cloud Function (triggered by Cloud Scheduler or by a batch-complete Pub/Sub event) to query the summary table for the batch key and produce the aggregated report. The report can be written as a simple JSON/CSV/HTML into gs://reports/<batch-id>/validation-report-<timestamp>.json and also ingested into a BigQuery reporting table for dashboarding.

The aggregating function should compute totals (files processed, total rows, total invalid rows), error breakout (top N error types with counts and example rejects paths), SLA flags (e.g., invalid_rate > threshold) and actionable links (rejects file URIs, correlation ids). Include job-level metadata like schema_version and processing_time percentiles so stakeholders can quickly assess health. Also attach a compact sample of failing rows or a pointer to sample rejects for fast inspection.

Automate notifications: after the report is written, the aggregator function can publish a short summary to Slack or email with a link to the full report in GCS and the dashboard. If thresholds are breached (e.g., >1% invalid rows or critical error types), trigger a PagerDuty alert. Keep the report immutable and versioned so historical audits are possible.

For replay and remediation, provide an admin endpoint or small UI that can list batches, open a rejects file, and allow reprocessing of corrected rows. Instrument metrics for report generation time and success/failure and include a runbook for common remediation steps. This design makes each validation run auditable, searchable, and actionable exactly what the business expects and what interviewers look for.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

22. A file lands in GCS, must be validated, and then trigger a Composer DAG. How would you implement this with Cloud Functions?

Start with a Cloud Function that subscribes to GCS OBJECT_FINALIZE so it only runs when upload is complete. Keep the function's work minimal: quick filename/size/encoding checks, sample-parse the header to validate basic schema, and compute a provenance tuple (object path, generationId, crc32c). If the file fails quick checks, move it to a quarantine prefix and write a manifest row with the failure reason; don't try heavy validation inside the function.

If the file looks good, write a durable manifest record (BigQuery or Firestore) that records ingest_id, file path, generationId and initial status=VALIDATED. This manifest prevents double-processing and gives you an auditable place to resume or replay. Then trigger the Composer DAG by creating a DAG run using Composer's REST API (Airflow REST or the Composer REST endpoint) or by publishing a message to a Pub/Sub topic that the DAG is configured to consume. Passing parameters (ingest_id, file path, schema version, correlation_id) as DAG run config keeps everything traceable.

Prefer using Composer's booked-in mechanism (trigger DAG run or use a Pub/Sub sensor inside the DAG) rather than trying to embed heavy logic in the function. If you need immediate and parameterized runs, call the Composer/Airflow REST API from the function with the DAG id and conf. If you want looser coupling, publish to a dedicated Pub/Sub topic the DAG can have a lightweight starter task that pulls the message and validates again before doing heavy work. Either approach should include idempotency checks against the manifest.

Production needs: protect the function with least-privilege service account (ability to read GCS, write manifest, and call Composer), set conservative timeouts, and add structured logs and metrics (files validated, DAG starts, failures). Ensure the DAG itself reconfirms file integrity (head checksums) and updates the manifest with final processing status so the function's trigger and the DAG stay in sync. Also add a replay endpoint or admin UI that can re-trigger DAG runs for a given ingest_id.

Finally, implement alerts and SLAs: alert on failed validation rates, DAG failures, or manifest inconsistencies, and keep a runbook for manual replay. This split keeps the Cloud Function as a fast gatekeeper and Composer as the stateful orchestrator simple, observable, and safe for interviews.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

23. A Dataproc PySpark job must be triggered after raw data is available. How would you orchestrate this with Cloud Functions?

Use a Cloud Function triggered by the GCS OBJECT_FINALIZE event to perform quick validation and then hand off job submission. The function should check the filename pattern and do a lightweight head-check (crc32c or small sample parse). If the file is invalid, quarantine it and write a manifest row; if valid, write a manifest entry marking intent-to-process (ingest_id, file path, generationId). This manifest is your single source of truth for idempotency and replays.

Submit the Dataproc job from the Cloud Function using Dataproc Workflow Templates or the jobs.submit API. I prefer Workflow Templates for production because they let you parameterize input path, output path, date, and resource settings and they simplify retries and consistent configuration. The function calls the Dataproc API with the template name and parameters, passes labels (ingest_id, job_type), and records the returned job_id in the manifest for tracking. This keeps the function simple while letting Dataproc handle heavy compute.

Have a watcher or use Dataproc job callbacks (via Pub/Sub job notifications or polling) to observe job success/failure and update the manifest accordingly. On transient failures, the watcher can auto-retry up to a bounded number with exponential backoff; on permanent failures move the ingest to a quarantined state and notify the data owners with failure details and links to logs. Avoid blind retries from the trigger function itself use the manifest/job-state driven approach to make retry decisions deterministic.

Make jobs idempotent: ensure your PySpark job writes to a staging location keyed by ingest_id or job_id and performs atomic swaps (e.g., write new data to a table partition or staging path, then move/rename to production) or perform MERGE semantics in downstream BigQuery. Use job labels to filter logs and billing, and limit IAM so the Cloud Function can only submit jobs (no broad project-wide permissions).

Operationalize with monitoring and runbooks: collect metrics (job duration, failure rates, retry counts), add alerts for repeated failures or backlogs, and provide an admin path to re-run a template with modified parameters. This pattern (validate → manifest → submit template → watch → update manifest) is robust, easy to explain in interviews, and safe in production.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. You must connect a Pub/Sub → Cloud Function → BigQuery → Looker pipeline. How would you glue the services together?

Treat each step as a single responsibility and make state transitions explicit. Have Cloud Functions subscribe to the Pub/Sub topic and perform lightweight validation/normalization of incoming JSON events. The function should enrich with ingest metadata, attach a correlation_id, validate schema, and then write rows to a staging BigQuery table or to GCS staging files (depending on latency and volume). Use batch writes or the BigQuery Storage Write API for high throughput and include an ingest_id or insert_id for idempotency.

Once the data lands in the staging table, have a controlled promotion step: either a scheduled job, a BigQuery scheduled query, or a downstream Cloud Function that triggers a MERGE from staging into the production Looker-facing datasets. Make the promotion idempotent (MERGE keyed by natural key + ingest_id) so replays don't duplicate data. Keep production tables partitioned and clustered for performance; Looker will point at these production tables (or to derived PDTs) rather than the raw staging table.

Integrate with Looker by ensuring Looker has read access to the BigQuery dataset and by automating model refreshes when necessary. For Looker PDTs or cache refreshes, use the Looker API to trigger a PDT build or an explore cache clear after the data promotion completes; have the promotion job call Looker's API with the affected model/explore and correlation_id so dashboards reflect fresh data quickly. If you use streaming updates, consider smaller, more frequent PDT refreshes or incremental PDTs rather than full rebuilds.

Add observability and error handling: write status and lineage to an audit table (ingest_id → staging_rows → promoted_rows → looker_refresh_status → timestamps) so you can trace a Pub/Sub message through to Looker. Use DLQs for bad Pub/Sub messages, pushes rejects to GCS for manual review, and alert on promotion failures or Looker refresh errors. Also include job labels and correlation ids in logs so on-call can jump straight to the failing component.

Security and governance: use least-privilege service accounts (Cloud Function for Pub/Sub→BigQuery write, promotion service account for BigQuery DML and Looker API calls), encrypt secrets in Secret Manager (Looker API keys), and set dataset access controls so Looker only reads final datasets. This clear separation and audit trail make the flow reliable and explainable exactly the sort of architecture an interviewer wants to hear.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

25. Your Cloud Function fails intermittently due to API timeouts. How would you handle retries?

First, make retries safe and intentional. Don't rely on blind automatic retries decide whether the function should retry a failing call, and where that retry logic lives (the function itself, Pub/Sub redelivery, or an external retrier like Cloud Tasks). For transient API timeouts, prefer exponential backoff with jitter inside the function for the specific external call (eg. retry 3 times with 200ms→1s→2s + jitter) so you give the remote service a chance to recover without thundering-herd effects. Keep retries bounded so you don't blow past the function timeout.

Second, use the right delivery mechanism. If the function is triggered by Pub/Sub, configure the subscription retry/Dead Letter Topic behavior so messages that keep failing get moved to DLQ after N attempts instead of looping forever. If you need guaranteed ordered retries or delayed retries, use Cloud Tasks: push a retry task with a delay and controlled rate so you can retry without tying up the function execution. Cloud Tasks also gives you visibility into attempts and backoff.

Third, make the function resilient: set a sensible function-level timeout slightly larger than your expected latency (but not huge), implement circuit-breaker logic for the flaky API (if many calls time out, short-circuit further attempts for a cooldown window), and add fallbacks (cached results, degraded behavior) when the external API is down. Also add instrumentation to measure timeout rates, latency percentiles, and retry counts so you can tune policies.

Fourth, ensure idempotency so retries are safe. Combine retries with an idempotency key (message_id/ingest_id) and design downstream effects to be repeatable or deduplicated (eg. write to staging and MERGE, or use insert_id for BigQuery streaming). That way both automatic and manual retries won't create duplicate side-effects.

Finally, notify and operate: emit structured logs and a retry metric, set alerts for rising timeout/retry rates, and capture sample failing payloads (sent to a secure DLQ) for debugging. With bounded retries, circuit-breaker protections, and strong observability, intermittent timeouts become manageable rather than catastrophic.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

26. Duplicate messages are causing duplicate loads in BigQuery. How would you make your function idempotent?

Start by attaching a strong, unique id to every incoming event use the native message_id from Pub/Sub or generate an ingest_id derived from source+generationId. Persist that id in a durable manifest (BigQuery/Firestore) or include it as insertId when using BigQuery streaming/Storage Write API so BigQuery can dedupe within a short window. Treat that id as the single source of truth for dedupe checks.

Second, avoid single-row, best-effort writes. Use a staging table or files in GCS with the ingest_id column and then run a deterministic MERGE into the final table keyed on the natural business key plus ingest_id. MERGE semantics allow you to upsert or ignore duplicates safely and make replays idempotent. For streaming, set insertId per row so BigQuery streaming can dedupe recent retries; for Storage Write API use provided dedupe keys or implement dedupe during the write.

Third, make the function manifest-driven: before processing, try to insert a manifest record with status=PENDING using an atomic operation (for example, a BigQuery insert with unique partition key or a conditional write in Firestore). If a manifest entry already exists for that ingest_id and shows SUCCESS, skip processing. Record transitions (PENDING→RUNNING→SUCCESS/FAILED) to handle mid-flight retries and to support safe replays. This prevents races and double-submits.

Fourth, design downstream operations to be idempotent as well. Avoid destructive replace semantics without checks prefer write-to-staging + atomic swap or MERGE. If you must call downstream APIs that are not idempotent, wrap them with an idempotency layer (store call results keyed by ingest_id and return cached response on retries). Log the ingest_id in every external call so troubleshooting duplicates is straightforward.

Finally, test and monitor: add tests that simulate duplicate deliveries and ensure the pipeline doesn't double-load. Emit metrics for duplicate-detected counts and alert on abnormal duplicate rates. With unique ids, manifest-driven state, and merge-based writes, you get robust idempotency that survives retries and replays.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. You need to monitor all pipeline errors centrally. How would you configure logging and alerts for Cloud Functions?

Centralize structured logs first. Ensure every Cloud Function emits structured JSON logs with fields like severity, timestamp, correlation_id, ingest_id, component, error_code, and contextual metadata. Use Cloud Logging (Stackdriver) as the central sink and standardize log keys so downstream log-based metrics and queries are reliable. Also attach trace IDs for distributed tracing across Pub/Sub → Function → BigQuery.

Create log-based metrics and dashboards. From the structured logs, create log-based metrics for error counts, exception types, DLQ moves, and validation-failure rates. Use those metrics to populate a centralized monitoring dashboard (Cloud Monitoring) that shows error trends, per-function latency percentiles, and backlog (Pub/Sub) or manifest lag. Dashboards make it easy for SREs and data owners to spot problems quickly.

Set up alerting policies on the most important signals. Create alerts for high error-rate (error count / minute above threshold), rising retry counts, DLQ volume, long-running function durations, and growing Pub/Sub backlog. Configure alert channels into on-call systems (PagerDuty), team Slack, and email, and craft alert messages that include the correlation_id, sample error messages, and a link to log queries or the relevant manifest entry. Use grouped alerts to avoid noise and set meaningful thresholds tied to SLOs.

Aggregate errors for analysis and reporting. Export logs to BigQuery for ad-hoc analytics (root-cause trends, top producers, frequent error messages) and to a long-term storage bucket for compliance. Enable Error Reporting (or equivalent) to automatically surface frequent exceptions and stack traces. Also consider exporting critical alerts and incidents to an incident management tool so you can track remediation and RCA.

Finally, operationalize with runbooks and access control: write clear runbooks linked from alerts showing triage steps and common fixes, give read-only log access to stakeholders and write access only to the service accounts that need it, and periodically review alert thresholds to reduce noise. With structured logs, log-based metrics, targeted alerts, and an export pipeline for analysis, you'll have a centralized, actionable view of pipeline errors that on-call teams can rely on.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

28. A GCS → Cloud Function → BigQuery pipeline for daily sales data ingestion. How would you do it?

Start with a Cloud Function that listens to GCS OBJECT_FINALIZE on the raw sales bucket and performs fast deterministic checks: filename convention, date extraction, size limits and a quick header/sample parse to ensure the CSV/Parquet looks sane. If the file fails those checks, copy it to a quarantine prefix and write an error record to an audit table fail fast and surface the problem rather than letting bad data reach downstream. The function's role is a gatekeeper, not the full transform.

If the file passes, write a manifest row (BigQuery or Firestore) with ingest_id, source_file, generationId, expected_date and status=PENDING. Then hand off processing by publishing a message to a dedicated Pub/Sub topic or by invoking a batch worker (Cloud Run or Dataflow) based on file size and complexity. For daily batch ingestion you'll usually prefer a simple pattern: function writes metadata and either directly kicks a GCS→BigQuery load job or signals a batch runner to do the heavy lifting.

For loading into BigQuery prefer load jobs from GCS for daily files they are cheaper and atomic. The worker should write cleaned Parquet/Avro to a staging bucket (if transformation required) and then run a GCS→BigQuery load into a staging table. After successful load, run an idempotent MERGE from staging into the production sales table partitioned by sale_date and clustered by relevant keys (store_id, product_id) so queries are efficient and cost-controlled. Record job_id and status back into the manifest.

Add post-processing steps: once MERGE succeeds update manifest to SUCCESS, move original raw file to an archive bucket with copy-verify-delete, and emit metrics (files processed, rows, bytes, failures). Keep a small replay/admin endpoint to re-run ingestion for a specific ingest_id and include audit columns (ingest_time, source_file, process_id) so any load can be traced and recreated.

Operationalize with monitoring and security: use least-privilege service accounts for the function and batch runner, create alerts for job failure or growing manifest backlogs, and produce a nightly report that shows ingest success rates and data freshness. This pattern separates concerns, provides idempotency and auditability, and is easy to explain in an interview.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

29. A Pub/Sub → Cloud Function → BigQuery pipeline for real-time fraud detection. How would you design it?

Design for low-latency, resiliency and safe enrichment. Have producers publish transaction events to a Pub/Sub topic; a lightweight Cloud Function subscribes to that topic, validates and normalizes the JSON, attaches correlation/ingest ids, and enriches with small cached lookups (e.g., user risk score from Memorystore). Keep synchronous work minimal anything heavier goes to an async stage. The function then publishes enriched events to a "scored" topic or directly buffers to a streaming sink.

For fraud detection, use two parallel paths: a fast path for real-time scoring and a slow path for heavier analytics. The fast path is the Cloud Function (or Cloud Run if you need more control) that calls a low-latency model endpoint (AI Platform/Vertex deployed model) or an internal scoring service and writes the score to BigQuery streaming/staging table with an insertId or to BigQuery Storage Write API in batched mode. The function should apply business thresholds and immediately emit alerts or webhook calls for high-risk transactions.

The slow path batches events into a staging area (GCS or BigQuery staging table) for more expensive feature enrichment, aggregation and model retraining signals. Use Dataflow or a scheduled batch job to compute rolling features and feed them back to the feature store or BigQuery for offline model training. Keep the real-time scoring idempotent by using message ids and by deduping during downstream MERGE operations.

Handle failures and DLQs carefully: configure Pub/Sub with a DLQ for poison messages, log invalid transactions to a rejects table for investigation, and add retry/backoff for transient scoring API timeouts (bounded, with circuit-breaker). Monitor key metrics: end-to-end latency (Pub/Sub → score → BigQuery), DLQ rate, model response errors, and alert on anomalies.

Finally, secure and govern the pipeline: use least-privilege service accounts, keep model keys in Secret Manager, label BigQuery tables and jobs for cost tracking, and provide a replay mechanism so suspicious batches can be re-scored after model updates. This hybrid fast/slow architecture balances latency needs with robust feature engineering and is easy to describe in an interview.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. Your ML team needs feature files validated and staged in GCS before training in Vertex AI. How would you use Cloud Functions here?

Use Cloud Functions as the first line of validation and orchestration. When feature files land in the designated staging bucket have a function triggered by OBJECT_FINALIZE do quick checks filename schema, partition/date correctness, checksum verification, and sample-parsing to confirm schema and data types. Surface any immediate failures by moving files to a quarantine prefix and recording a validation failure record in a feature_manifest table with error reasons so data engineers can fix the source.

For files that pass basic checks, run a deeper validation step: either inside a Cloud Function (for small files) or by publishing a job message to Pub/Sub for a Cloud Run/Dataflow validator for large files. Deep validation includes full-schema conformance, cardinality checks, distribution checks (null rates, distinct counts), feature ranges, and ensuring no leakage features exist. The validator should write a validation report and summary metrics into BigQuery and write cleaned feature Parquet/TFRecord outputs to a ready/ staging prefix.

Record the validation outcome in a manifest row with status=READY and any metrics (rows, null_rate, histograms). The Cloud Function or orchestrator then triggers the Vertex AI training workflow or registers the ready dataset with Vertex AI Feature Store / Vertex data ingestion process for example by calling Vertex AI APIs or by creating a trigger message that the ML training pipeline watches. Include provenance metadata (source_file, generationId, validation_report_uri) so the ML team can trace which features were used.

Provide operational tooling: a small admin endpoint or UI to re-run validation for a given ingest_id, a replay path to re-stage corrected files, and automated notifications to the ML team with links to the validation report. Implement retention and access controls so sensitive feature data is secured and audited; store secrets in Secret Manager and encrypt objects.

Finally, monitor and automate quality gates: block training runs unless the manifest shows status=READY and validation thresholds are met; surface drift/quality metrics over time and alert on rising null rates or schema changes. This pattern puts Cloud Functions in the gatekeeper/orchestrator role, uses scalable workers for heavy validation, and gives Vertex AI the clean, validated inputs ML teams need a practical, interview-friendly design.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar